Danmarks Tekniske Universitet

MODELING CO2 CONCENTRATIONS USING HIDDEN MARKOV MODELS

2026

Special Course in Statistical Modelling

Mads Holme Håkonsson
s234936

Supervisor
Jan Klobbenborg Møller

31. maj 2026

AI Declaration

AI tools has been used for formatting LaTex figures, tables and equations and grammar checking in the report. No AI has been used for for writing the math or the academic content of the report or the code. No Agentic AI tools has been used for wrriting the src code or the report. Agentic AI tools has been used for writing some of testing files in the tests directory or refactoring code as renaming files, variables or functions.

Indhold

1 Introduction

Hidden Markov Models (HMMs) are a powerful tool for modeling sequential data, where the underlying states of the system are not directly observable. They have been widely used in various fields such as speech recognition, natural language processing, and bioinformatics. [6].
The goal of this project is to inference if a person is in a room based on CO_2 data collected from a sensor in the room. To do this, we will use a Hidden Markov Model to model the relationship between the observed CO_2 levels and the hidden states of whether a person is in the room or not and if a window is open or not. When training the models, we will use `JAX` [2] and `EQUINOX` [4] for automatic differentiation and optimization, and we will use `Optax` [3] as our optimization library.
WRITE HERE THE RESULTS YOU FIND LATER ON.

This report is written for the purpose of given a concise outline of the projects achivements, design choices, model results and interpretaiton. The report is written for future students to be able to easily carry on the the project and extend it in some way.

2 Software

This section will will give a explanation of the classes implemented, the design choices made and the primarylibraries used. A repository link can be found in 8.1 and the envoriment details and short source code overview can be found in 8.2. Class diagrams can be found in 8.4.

2.1 Class overview

The source code is organized into 6 main components: the `HMM` class, the `Emissions` class, the `Transitions` class, the `HMMParams` class, the `Inference` class, and the `Solvers` class. All classes except `HMM` are abstract interfaces. The `HMM` class acts as the main entry point, providing a simple interface for training and inference by delegating to the other components. The key design principle is that concrete implementations of `Emissions` and `Transitions` are injected at runtime, allowing different HMM variants to be used interchangeably. A detailed explanation of each class and the motivation behind the design choices can be found in 8.5, and class diagrams are provided in 8.4.

2.2 Design Principles

An Object-Oriented Programming (OOP) design was chosen for this project to promote modularity, extensibility, and separation of concerns. The Dependency Inversion Principle (DIP) was central to the design of all core components. They are defined as abstract interfaces, so different implementations can be swapped in without affecting the rest of the system. This modularity is also motivated by the mathematical structure of HMMs themselves. The forward algorithm, and HMM inference in general, depends only on two quantities. The emission density $p(o_t \mid s_t)$ and the transition probability $p(s_t \mid s_{t-1})$. It is entirely agnostic to how those quantities are computed. By encapsulating each behind an abstract interface, different HMM variants can be used interchangeably without modifying the inference or optimization code, so modularity follows naturally from the structure of the model. This also makes it straightforward to extend the project by adding new emission distributions, transition models, or inference algorithms without modifying existing code.

2.3 Primary Libraries

The primary libraries used in this projects are `JAX` [2], `EQUINOX` [4], and `Optax` [3].
`JAX` was chosen over `NumPy` for its automatic differentiation capabilities and performance optimizations. However, `JAX` is designed for functional programming and does not natively support classes or object-oriented design. Therefore, `EQUINOX` was chosen as it's a library built on top of `JAX` that provides a simple and flexible interface for defining classes as pytrees. [1], `JAX` can then automatically differentiate trough the classes and the leaves of the pytree.
`Optax` was chosen as the optimization library for its seamless integration with `JAX` and `EQUINOX`, providing a wide range of optimization algorithms and utilities for training machine learning models. `Optax` was purely choosen as it's the default optimization library for `JAX` and there may be a better choice for this project, but it was not explored due to time constraints.

[1] A pytree is a nested data strucure in `JAX`:`https://docs.jax.dev/en/latest/pytrees.html`

3 Generel Theory

In this section, we will present the type of forward algorithm used to compute the likelihood and the pseudo residuals used to evaluate the model fit.

3.1 Forward Algorithm

The forward algorithm implemented is called recursive filtering [5] as it constructs the current statedistribution $p(X_t \mid Y_{1:t})$ recursively from the previous state distribution $p(X_t \mid Y_{1:t-1})$.

Algorithm 1 Recursive Filtering Forward Algorithm

Require: Observations $\mathbf{y} = (y_1, \ldots, y_T)$, covariates $\mathbf{x} = (x_1, \ldots, x_T)$, initial state distribution $\mathbf{u}_0 \in \mathbb{R}^{1\times N}$, HMM parameters θ

Ensure: Predicted distributions $\{\mathbf{u}_t\}$, filtered distributions $\{\mathbf{u}_{t|t}\}$, step likelihoods $\{f_t\}$

for $t = 1, \ldots, T$ **do**
  $\Gamma_t \leftarrow \textsc{TransitionMatrix}(\theta, t, y_t, x_t)$ ▷ $N \times N$ transition matrix
  $\mathbf{u}_{t|t-1} \leftarrow \mathbf{u}_{t-1|t-1} \cdot \Gamma_t$ ▷ Predicted state distribution, $1 \times N$
  $\mathbf{g}_t \leftarrow \textsc{Density}(\theta, t, y_t, x_t)$ ▷ Emission densities for all states, $1 \times N$
  $f_t \leftarrow \sum_{i=1}^{N} [\mathbf{u}_t \odot \mathbf{g}_t]_i$ ▷ Marginal likelihood at time t
  $f_t \leftarrow \max(f_t,\ 10^{-10})$ ▷ Clip to avoid division by zero
  $\mathbf{u}_{t|t} \leftarrow \dfrac{\mathbf{u}_t \odot \mathbf{g}_t}{f_t}$ ▷ Filtered state distribution (normalized)
end for
return $\{\mathbf{u}_t\},\ \{\mathbf{u}_{t|t}\},\ \{f_t\}$

where $\odot$ denotes element-wise multiplication of vectos.

The log-likelihood is obtained by summing the log of the step likelihoods:

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \log f_t \tag{1}$$

3.2 Forecast Pseudo Residuals

A method to access the goodness-of-fit of the HMM is to compute the pseudo residuals. For a introduction to pseudo residuals, see chapter 6.2 in [10].
The Forecast Pseudo Residuals used in this project compute the the inverse of the cummulative distribution function (CDF) of the one-step-ahead predictive distribution. If the model is is good, the pseudo residuals should be approximately standard normally distributed.

Briefly desribed, a forecast pseudo residual for a HMM is defined as

$$z_{t+1} = \Phi^{-1}(F_Y(y_{t+1} \mid Y_{1:t}))$$

where Φ^{-1} is the inverse CDF of the standard normal distribution and F_Y is the predictive CDF of Y_{t+1} given $Y_{1:t}$, defined as

$$F_Y(y_{t+1} \mid Y_{1:t}) = \sum_{i=1}^{N} u^i_{t+1|t} F_i(y_{t+1} \mid X = i)$$

where X is the hidden state.
Because z_{t+1} now should follow a standard normal distribution, we can use normality tests to assess the goodness-of-fit of the model.
In this project, we use the QQ-plot to visually assess the normality of the pseudo residuals.

3.3 Test Statistics

Other methods to assess the goodness of fit of the HMM model include the AIC [7] and BIC [8] test statistics. These are defined as

$$\text{AIC} = 2k - 2\mathcal{L}$$

$$\text{BIC} = k\log(n) - 2\mathcal{L}$$

where k is the number of parameters in the model, n is the number of observations and $\mathcal{L}$ is the log-likelihood of the model.
The AIC and BIC test statistics are used to compare different models, where a lower value initiates a better fit.
Hoever, a lot of the models implemented are nested models, so a Likelihood Ratio Test (LRT) [9] can also be used to compare the models. The LRT test statistic is defined as

$$\text{LRT} = 2(\mathcal{L}_1 - \mathcal{L}_0)$$

where $\mathcal{L}_1$ is the log-likelihood of the more complex model and $\mathcal{L}_0$ is the log-likelihood of the simpler model.
The LRT test statistic follows a chi-squared distribution with degrees of freedom equal to the difference in the number of parameters between the two models.

4 Models & Results

For generel theory on HMM's, see [10] and [5] chapter 11. All the models implemented are 4 state Gaussian HMMs (emission density is a Gaussian distribution) and we have kept the the mean value of state 1 fixed at 400 as this is the baseline CO_2 level in the room when no one is present and all windows are closed. This is also shown in the figure 6.

4.1 Ordinary HMM model

4.1.1 Theory

The ordinary HMM model is the simplest model implemented in this project and the one most commonly used in the literature. The transition probabilities are constant and do not depend on time, the observations or any covariates, and so does the emission density.
For this model, we want to estimate the parameters $\theta = \{\Gamma, \mu, \sigma^2\}$ where $\Gamma \in \mathbb{R}^{n\times n}$ is the transition matrix, $\mu \in \mathbb{R}^n$ is the mean vector of the Gaussian emission density and $\sigma^2 \in \mathbb{R}^n$ is the variance vector of the Gaussian emission density. Because the markov chain is finite, discrete and homogeneous, a unique strictly positive stationary distribution δ exists (chappter 1.3.2 [10]) and is given by

$$\delta = \mathbf{1}^T(I - \Gamma + E)^{-1}$$

where E is a matrix of ones.

To handle the natural bounds on the standard deviation parameters and transition proberbilities, the parameters were reparameterized as follows:

$$\sigma_i^2 = \exp(\eta_i)$$

For the transition matrix, only the $n(n-1)$ off-diagonal entries are parametrized; the diagonal is used as the reference category with log-odds fixed to zero. Concretely, the unconstrained parameters are stored as a matrix $\xi \in \mathbb{R}^{n\times(n-1)}$ that holds the off-diagonal logits. The transition matrix is then obtained by exponentiating these logits in the off-diagonal positions of an identity matrix (so the diagonal is $\exp(0) = 1$) and row-normalizing:

$$\Gamma_{ij} = \begin{cases} \dfrac{\exp(\xi_{ij})}{1 + \sum_{k\neq i}\exp(\xi_{ik})}, & j \neq i, \\ \dfrac{1}{1 + \sum_{k\neq i}\exp(\xi_{ik})}, & j = i. \end{cases}$$

This is equivalent to a row-wise softmax with the diagonal entry pinned as the reference, and guarantees $\Gamma_{ij} \geq 0$ and $\sum_j \Gamma_{ij} = 1$ for any $\xi \in \mathbb{R}^{n\times(n-1)}$. The stationary distribution δ is not parametrized directly; it is recomputed from Γ at every optimization step via $\delta = \mathbf{1}^T(I - \Gamma + E)^{-1}$, so reparametrizing Γ through ξ implicitly reparametrizes δ.

The unconstrained parameters $\eta \in \mathbb{R}^n$ and $\xi \in \mathbb{R}^{n\times(n-1)}$ are the quantities that are optimized during the fitting process.

The μ parameters are not bounded, but to avoid label switching for the gaussitan distrbitions, the mean parameters are ordered as $\mu_1 < \mu_2 < \mu_3 < \mu_4$ by the reparameterization

$$\mu_1 = \zeta_1$$

$$\mu_i = \zeta_1 + \sum_{k=2}^{i}\exp(\zeta_k)$$

where $\zeta \in \mathbb{R}^n$ is the unconstrained parameter vector that is optimized during the fitting process.

4.1.2 Results

The model was fitted on the CO_2 observations by minimising the negative log-likelihood with L-BFGS, with μ_1 frozen at 400 ppm as discussed above. The estimated transition matrix is shown in Table 1 and the estimated emission parameters in Table 2.

	State 1	State 2	State 3	State 4
State 1	0.8496	0.1476	0.0029	0.0000
State 2	0.0828	0.8486	0.0686	0.0000
State 3	0.0004	0.1605	0.6841	0.1550
State 4	0.0000	0.0000	0.0718	0.9282

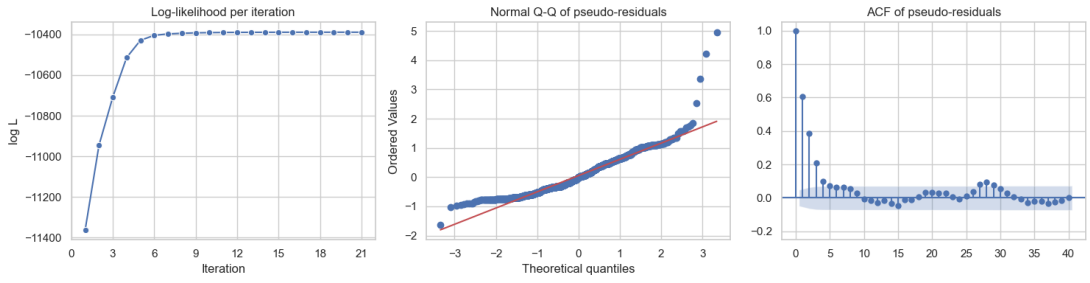
Tabel 1: Estimated transition matrix for the ordinary 4-state HMM.

The transition matrix has a strongly dominant diagonal, with self-transition probabilities between 0.68 and 0.93, so each regime is persistent once entered. The off-diagonal mass is concentrated on the immediate neighbours of each state: the probability of jumping between non-adjacent states (e.g. $1 \leftrightarrow 3$, $1 \leftrightarrow 4$, $2 \leftrightarrow 4$) is essentially zero. The chain therefore moves through the four CO_2 levels in a band-diagonal fashion rather than jumping directly from baseline to a high regime.

State	Mean ($\hat{\mu}$)	Std. Dev. ($\hat{\sigma}$)
State 1	400.000	21.748
State 2	535.684	76.573
State 3	889.928	110.981
State 4	1306.484	197.410

Tabel 2: Estimated emission parameters (Gaussian) for the ordinary 4-state HMM.

The four estimated state means in Table 2 (400, 536, 890, 1306 ppm) admit a natural interpretation as a *baseline*, *low*, *medium* and *high* CO_2 regime. The emission standard deviations grow with the mean, which is consistent with the higher-occupancy regimes being noisier (more people, more variable activity in the room).



Figur 1: Diagnostic plots for the ordinary 4-state HMM. Left: log-likelihood per iteration. Centre: normal Q–Q plot of pseudo-residuals. Right: ACF of pseudo-residuals.

Figure 1 summarises the fit. The log-likelihood trace (left panel) shows that the optimiser converges quickly towards the minimum of the negative log-likelihood: almost all of the improvement

happens in the first few iterations and the curve then plateaus. The Q–Q plot (centre) shows that the pseudo-residuals are approximately normal in the bulk, with some deviation in the tails. The ACF of the pseudo-residuals (right) is, however, clearly non-trivial: the autocorrelation decays only slowly and stays well above the confidence band for many lags. This indicates that the observations carry time-series structure — autoregressive behaviour of the CO_2 signal — that the ordinary HMM does not capture, and motivates the AR extensions considered in the following subsections.

4.2 Autoregressive HMM

4.2.1 Theory

The Autoregressive HMM (AR-HMM) extends the ordinary HMM by letting the emission distribution depend on the previous observation. Conditional on being in state i at time t, the observation is modelled as

$$Y_t \mid S_t = i,\ Y_{t-1} = y_{t-1} \ \sim\ \mathcal{N}\big(\mu_i + \phi_i(y_{t-1} - \mu_i),\ \sigma_i^2\big)\,,$$

so each state carries its own AR(1) coefficient $\phi_i \in (-1, 1)$ in addition to a mean and variance. This captures the within-regime temporal correlation of the CO_2 signal that the ordinary HMM left in the pseudo-residuals. The transition matrix Γ is parametrised exactly as in the ordinary HMM, and the AR coefficients are reparametrised via $\phi_i = \tanh(\psi_i)$ to enforce the stationarity constraint $|\phi_i| < 1$. The full parameter vector to be estimated is $\theta = \{\Gamma,\ \mu,\ \sigma^2,\ \phi\}$.

4.2.2 Results

The AR-HMM was fitted on the same CO_2 observations, again with μ_1 frozen at 400 ppm and using L-BFGS on the negative log-likelihood. The ordinary-HMM estimates were used as a warm start for Γ, μ and σ, and the AR coefficients were initialised at $\phi_i = 0.2$ for all states. The estimated transition matrix is shown in Table 3 and the estimated emission parameters in Table 4.

	State 1	State 2	State 3	State 4
State 1	0.7521	0.1685	0.0016	0.0779
State 2	0.2698	0.5960	0.0633	0.0709
State 3	0.0516	0.1027	0.8457	0.0000
State 4	0.0067	0.0492	0.0153	0.9289

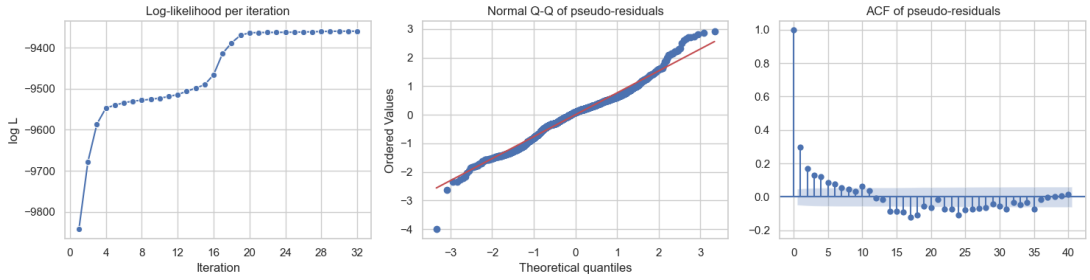
Tabel 3: Estimated transition matrix for the AR-HMM.

The transition matrix remains diagonally dominant but is noticeably less banded than for the ordinary HMM: non-neighbour transitions (e.g. $1 \to 4$, $2 \to 4$, $3 \to 1$) now carry small but non-zero probabilities, while the self-transition probabilities of the intermediate states have dropped (state 2 from 0.85 to 0.60, state 3 stays at 0.85). Because the within-state AR term already explains a portion of the persistence in the signal, the chain no longer needs as strong self-loops to fit the data.

The four state means in Table 4 (400, 509, 560, 1367 ppm) again separate a baseline, two low-to-medium regimes and a high regime, although states 2 and 3 are now much closer together than in the ordinary HMM. All four AR coefficients are positive and substantial, ranging from $\hat{\phi}_2 = 0.37$ up to $\hat{\phi}_3 = 0.95$; the high-CO_2 state (state 4) also has a strong AR component ($\hat{\phi}_4 = 0.83$). This confirms that a non-trivial amount of the temporal structure in the observations is within-regime autocorrelation rather than regime switching.

State	Mean ($\hat{\mu}$)	Std. Dev. ($\hat{\sigma}$)	AR coeff. ($\hat{\phi}$)
State 1	400.000	12.501	0.6235
State 2	509.326	77.704	0.3659
State 3	560.287	11.006	0.9462
State 4	1366.661	102.836	0.8318

Tabel 4: Estimated emission parameters for the AR-HMM (Gaussian with AR(1) residuals).



Figur 2: Diagnostic plots for the AR-HMM. Left: log-likelihood per iteration. Centre: normal Q–Q plot of pseudo-residuals. Right: ACF of pseudo-residuals.

Figure 2 shows the diagnostics for the AR-HMM. As for the ordinary HMM, the optimiser converges quickly toward the minimum of the negative log-likelihood. The Q–Q plot of the pseudo-residuals is closer to the diagonal than in the ordinary case, and — most importantly — the ACF of the pseudo-residuals is now much closer to white noise: the strong, slowly decaying autocorrelation observed for the ordinary HMM has largely been absorbed by the AR(1) emission, leaving only a few small spikes outside the confidence band.

4.3 Autoregressive HMM of order 2

4.3.1 Theory

The AR(2)-HMM extends the AR-HMM by letting the within-state autoregressive component depend on the *two* most recent observations rather than just one. Conditional on being in state i at time t, the observation is modelled as

$$Y_t \mid S_t = i,\ y_{t-1}, y_{t-2} \ \sim\ \mathcal{N}\big(\mu_i + \phi_{i,1}(y_{t-1} - \mu_i) + \phi_{i,2}(y_{t-2} - \mu_i),\ \sigma_i^2\big)\,,$$

so each state now carries a pair of AR coefficients $(\phi_{i,1}, \phi_{i,2})$ in addition to a mean and variance. This allows the model to capture richer within-regime dynamics — in particular a non-monotone impulse response of the CO_2 signal — that an AR(1) emission cannot represent. The transition matrix and the mean/variance reparametrisations are the same as in the AR-HMM, and the full parameter vector is $\theta = \{\Gamma,\ \mu,\ \sigma^2,\ \phi_1,\ \phi_2\}$.

4.3.2 Results

The AR(2)-HMM was fitted on the same CO_2 observations with μ_1 frozen at 400 ppm and L-BFGS on the negative log-likelihood. The AR-HMM estimates were used as a warm start, and the second-order AR coefficients were initialised at $\phi_{i,2} = 0.2$ for all states. The estimated transition matrix is shown in Table 5 and the estimated emission parameters in Table 6.

The transition matrix is very close to the one obtained for the AR-HMM: the diagonal dominance is preserved and the off-diagonal pattern is essentially unchanged. Adding a second AR lag

	State 1	State 2	State 3	State 4
State 1	0.7115	0.2065	0.0000	0.0820
State 2	0.2848	0.5479	0.1055	0.0618
State 3	0.0216	0.1325	0.8459	0.0000
State 4	0.0061	0.0453	0.0171	0.9315

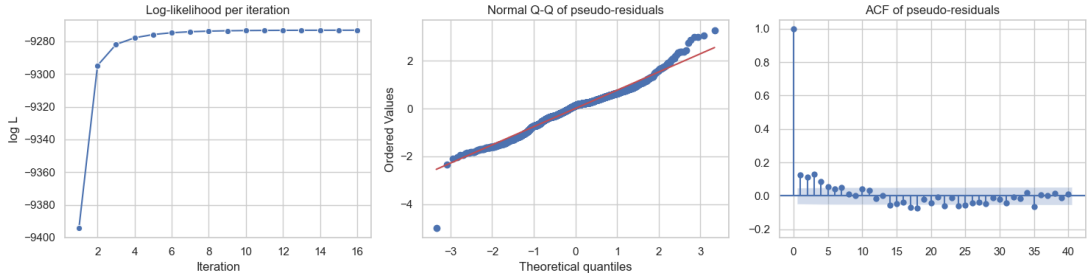
Tabel 5: Estimated transition matrix for the AR(2)-HMM.

therefore does not noticeably change the regime-switching structure — it mainly refines the within-state dynamics.

State	Mean ($\hat{\mu}$)	Std. Dev. ($\hat{\sigma}$)	AR coeff. ($\hat{\phi}_1$)	AR coeff. ($\hat{\phi}_2$)
State 1	400.000	10.765	0.7027	−0.0781
State 2	525.630	72.083	0.5941	−0.2109
State 3	716.015	16.158	0.9596	0.0565
State 4	1299.660	97.603	0.9997	−0.2093

Tabel 6: Estimated emission parameters for the AR(2)-HMM (Gaussian with AR(2) residuals).

The first-order AR coefficients in Table 6 remain strongly positive across all states (0.59–1.00), while the second-order coefficients are small and mostly negative, ranging from −0.21 to 0.06. This small negative $\phi_{i,2}$ partly corrects the overshoot of the AR(1) component and produces a slightly damped, more realistic impulse response of the CO_2 signal. Note that state 4 reaches $\hat{\phi}_{4,1} \approx 1$, which is on the edge of the stationarity region for the AR(2) process; this suggests the high-CO_2 regime behaves close to a unit-root process and could in principle be modelled with a non-stationary component.



Figur 3: Diagnostic plots for the AR(2)-HMM. Left: log-likelihood per iteration. Centre: normal Q–Q plot of pseudo-residuals. Right: ACF of pseudo-residuals.

Figure 3 shows the diagnostics for the AR(2)-HMM. The log-likelihood again converges quickly to its optimum, and the Q–Q plot of the pseudo-residuals is similar to that of the AR-HMM. The ACF of the pseudo-residuals is essentially indistinguishable from the AR-HMM case: most lags lie within the confidence band, with only a few small residual spikes. In other words, going from order 1 to order 2 yields only a marginal improvement in the residual diagnostics, indicating that an AR(1) emission already captures the bulk of the within-regime temporal structure in the CO_2 signal.

4.4 Second-order HMM

4.4.1 Theory

The second-order HMM relaxes the Markov assumption on the latent chain: the next state depends on the *two* most recent latent states rather than just one. The transition probabilities are therefore indexed by a pair of states,

$$\Gamma_{(i,j),k} \;=\; P(S_{t+1} = k \mid S_{t-1} = i,\, S_t = j)\,,$$

which gives an $n^2 \times n$ tensor of conditional probabilities (still row-normalised in k). The emission density is again Gaussian, with the mean depending only on the current state $\mu(s_t)$, but the standard deviation allowed to depend on *both* the previous and the current state: $\sigma(s_{t-1}, s_t)$. Operationally, the second-order chain is handled by augmenting the state space to pairs (s_{t-1}, s_t) and applying the standard forward recursions on the augmented chain. The full parameter vector to be estimated is $\theta = \{\Gamma_{(\cdot,\cdot),\cdot},\, \mu,\, \sigma^2\}$, with the same row-normalised softmax / log-variance reparametrisations as before.

4.4.2 Results

The second-order HMM was fitted on the same CO_2 observations, with μ_1 frozen at 400 ppm. The estimated transition tensor is shown in Table 7 and the estimated emission parameters in Table 8.

From (s_{t-1}, s_t)	$s_{t+1}=0$	$s_{t+1}=1$	$s_{t+1}=2$	$s_{t+1}=3$
$(0,0)$	0.7200	0.2100	0.0700	–
$(0,1)$	–	1.0000	–	–
$(0,2)$	–	0.3200	0.6800	–
$(0,3)$	0.8000	–	–	0.2000
$(1,0)$	1.0000	–	–	–
$(1,1)$	0.0700	0.7900	0.1100	0.0300
$(1,2)$	–	–	0.9300	0.0700
$(1,3)$	–	–	–	1.0000
$(2,0)$	0.1000	–	0.1100	0.8000
$(2,1)$	0.0500	0.2800	0.6600	–
$(2,2)$	–	0.8700	–	0.1300
$(2,3)$	–	–	–	1.0000
$(3,0)$	–	–	–	1.0000
$(3,1)$	1.0000	–	–	–
$(3,2)$	–	1.0000	–	–
$(3,3)$	–	–	0.0500	0.9400

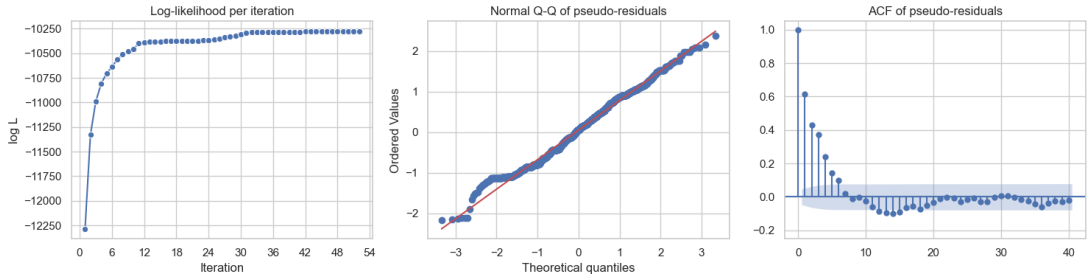
Tabel 7: Estimated transition probabilities for the second-order HMM. Each row is a state pair (s_{t-1}, s_t); columns give the next-state probability $P(s_{t+1} \mid s_{t-1}, s_t)$. A dash (−) indicates zero or negligible probability.

The second-order transition table shows that for most state pairs the next state is essentially determined by the current state s_t, with only mild dependence on the previous state s_{t-1} — e.g. rows $(1,1)$, $(2,2)$ and $(3,3)$ all assign most of the mass to staying in the current state, as in the ordinary HMM. Some pairs, however, are concentrated on a single successor (rows showing probability 1), which reflects rare or transient two-step patterns where the fit has very little data: these entries should be read as data-driven artifacts rather than as strong dynamical statements.

		Std. Dev. $\hat{\sigma}(s_{t-1})$			
s_t	Mean $\hat{\mu}$	$s_{t-1}=0$	$s_{t-1}=1$	$s_{t-1}=2$	$s_{t-1}=3$
0	400.000	6.377	5.710	6.943	1881.525
1	443.148	22.545	25.812	151.388	270.679
2	582.056	115.027	76.082	62.643	227.904
3	1197.129	1072.051	418.213	391.613	200.235

Tabel 8: Estimated emission parameters for the second-order HMM. The emission mean $\hat{\mu}$ depends only on the current state s_t, while the standard deviation $\hat{\sigma}$ depends on both the previous state s_{t-1} and the current state s_t.

The emission means in Table 8 again order the four states into a baseline / low / medium / high CO_2 regime. The standard deviations, which now depend on the pair (s_{t-1}, s_t), vary substantially across previous states: in particular, several σ values are very large (e.g. 1881 ppm for $s_t = 0$ coming from $s_{t-1} = 3$, or 1072 ppm for $s_t = 3$ coming from $s_{t-1} = 0$). These large variances sit on transitions that are essentially never observed, so the corresponding σ values are only weakly identified — the optimiser is free to inflate them without affecting the likelihood.



Figur 4: Diagnostic plots for the second-order HMM. Left: log-likelihood per iteration. Centre: normal Q–Q plot of pseudo-residuals. Right: ACF of pseudo-residuals.

Figure 4 shows the diagnostics for the second-order HMM. The log-likelihood converges quickly toward the minimum of the negative log-likelihood, in the same way as for the previous models. The Q–Q plot of the pseudo-residuals is comparable to the ordinary HMM, and the ACF of the residuals still shows a slow decay with several lags well outside the confidence band. The second-order extension therefore improves the flexibility of the latent dynamics, but — because it does not introduce any within-state autocorrelation in the observations — it does not remove the residual time-series structure in the same way the AR-HMM does.

4.5 AR(2) second-order HMM

4.5.1 Theory

The AR(2) second-order HMM combines the two previous extensions: the latent chain is second-order (transitions depend on the pair (s_{t-1}, s_t)) and the emission density carries an AR(2) component whose parameters also depend on the pair (s_{t-1}, s_t). Conditional on $(S_{t-1}, S_t) = (i, j)$ and the two most recent observations, the observation density is

$$Y_t \mid (i,j),\ y_{t-1}, y_{t-2} \ \sim\ \mathcal{N}\Big(\mu_j + \phi_{(i,j),1}(y_{t-1} - \mu_j) + \phi_{(i,j),2}(y_{t-2} - \mu_j),\ \sigma^2_{(i,j)}\Big)\,.$$

The mean depends only on the current state s_t, while $\sigma^2_{(i,j)}$ and the AR coefficients $(\phi_{(i,j),1}, \phi_{(i,j),2})$ are pair-specific. The transition tensor is the same as in the second-order HMM, and the same

reparametrisations are used for the transition rows, the log-variances and the AR coefficients. The full parameter vector to be estimated is $\theta = \{\Gamma_{(\cdot,\cdot),\cdot}, \mu, \sigma^2, \phi_1, \phi_2\}$.

4.5.2 Results

The AR(2) second-order HMM was fitted on the same CO_2 observations, with μ_1 frozen at 400 ppm and the previous models used as a warm start. The estimated transition tensor is shown in Table 9 and the estimated emission parameters in Table 10.

From (s_{t-1}, s_t)	$s_{t+1}=0$	$s_{t+1}=1$	$s_{t+1}=2$	$s_{t+1}=3$
$(0,0)$	0.6200	0.0900	0.2900	–
$(0,1)$	–	1.0000	–	–
$(0,2)$	–	0.5000	0.2900	0.2100
$(0,3)$	0.2400	–	–	0.7600
$(1,0)$	1.0000	–	–	–
$(1,1)$	0.2500	0.4900	0.1300	0.1300
$(1,2)$	–	0.0600	0.9300	0.0100
$(1,3)$	–	–	–	1.0000
$(2,0)$	–	–	0.9900	0.0100
$(2,1)$	0.1300	0.6700	0.1900	–
$(2,2)$	–	0.2200	0.7400	0.0400
$(2,3)$	–	–	–	1.0000
$(3,0)$	–	–	–	1.0000
$(3,1)$	1.0000	–	–	–
$(3,2)$	–	1.0000	–	–
$(3,3)$	0.0900	0.0100	0.0700	0.8300

Tabel 9: Estimated transition probabilities for the AR(2) second-order HMM. Each row is a state pair (s_{t-1}, s_t); columns give the next-state probability $P(s_{t+1} \mid s_{t-1}, s_t)$. A dash (−) indicates zero or negligible probability.

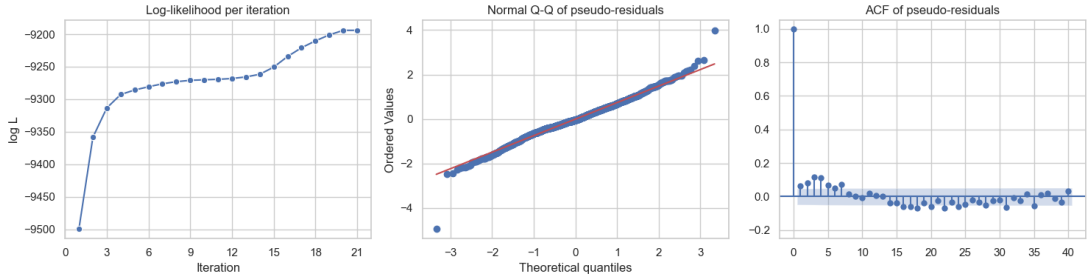
The transition tensor follows the same overall pattern as the second-order HMM: rows like $(1,1)$, $(2,2)$ and $(3,3)$ keep most of their mass on the current state, while several rare two-step pairs end up with all the mass on a single successor. The presence of an AR(2) emission shifts some mass off the diagonal compared to the second-order HMM (e.g. $(0,0)$ now has $P(s_{t+1}=2 \mid 0,0) = 0.29$ versus essentially zero before), because the within-state autocorrelation already explains part of the persistence and the latent chain no longer needs as strong self-loops.

Table 10 reports the pair-specific emission parameters. The four state means again order the regimes (400, 489, 710, 1399 ppm). The first-order AR coefficients $\hat{\phi}_1$ are uniformly large and positive (most are between 0.6 and 1.0), confirming that within-regime autocorrelation dominates the short-term dynamics regardless of s_{t-1}. The second-order coefficients $\hat{\phi}_2$ are mostly small and negative, in line with what was observed for the simpler AR(2)-HMM. The last column reports $\hat{\phi}_1 + \hat{\phi}_2$, which approximates the long-run persistence of the AR(2) component: a few pairs — notably $(1,3)$ with $\hat{\phi}_1 + \hat{\phi}_2 = 1.23$ — fall outside the stationarity region. These non-stationary pair-specific AR processes sit on transitions that are sparsely populated in the data and should be interpreted as weakly identified artifacts of the fit, in the same way as the inflated σ values in the second-order HMM.

Figure 5 shows the diagnostics for the AR(2) second-order HMM. The log-likelihood again converges quickly to its optimum. The Q–Q plot of the pseudo-residuals is close to the diagonal, and the ACF is essentially white noise: combining a second-order chain with an AR(2) emission

s_{t-1}	s_t	$\hat{\mu}$	$\hat{\sigma}$	$\hat{\phi}_1$	$\hat{\phi}_2$	$\hat{\phi}_1 + \hat{\phi}_2$
0	0	400.000	13.440	0.6692	−0.0675	0.6017
1	0	400.000	1.570	0.7573	0.0034	0.7607
2	0	400.000	7.673	0.8352	−0.1693	0.6658
3	0	400.000	252.443	1.0000	−0.1515	0.8485
0	1	488.949	32.808	0.5090	−0.2263	0.2827
1	1	488.949	72.020	0.6456	−0.1660	0.4796
2	1	488.949	30.655	0.9208	0.0584	0.9792
3	1	488.949	36.571	0.2970	−0.1088	0.1882
0	2	710.357	0.049	0.9904	−0.5889	0.4015
1	2	710.357	67.830	0.9244	−0.1207	0.8036
2	2	710.357	8.051	1.0000	−0.0011	0.9989
3	2	710.357	105.328	0.9904	−0.7683	0.2221
0	3	1399.061	141.474	1.0000	−0.0611	0.9389
1	3	1399.061	457.697	0.2307	0.9950	1.2257
2	3	1399.061	173.760	0.5703	0.3441	0.9145
3	3	1399.061	64.052	1.0000	−0.1992	0.8008

Tabel 10: Estimated emission parameters for the AR(2) second-order HMM. The emission mean $\hat{\mu}$ depends only on the current state s_t. The standard deviation $\hat{\sigma}$ and AR(2) coefficients $\hat{\phi}_1$, $\hat{\phi}_2$ depend on both (s_{t-1}, s_t). A sum $\hat{\phi}_1 + \hat{\phi}_2 > 1$ indicates a non-stationary AR process for that state pair.



Figur 5: Diagnostic plots for the AR(2) second-order HMM. Left: log-likelihood per iteration. Centre: normal Q–Q plot of pseudo-residuals. Right: ACF of pseudo-residuals.

gives the best residual diagnostics of the four models, even though the marginal improvement over the plain AR-HMM is small.

5 Discussion

6 Conclusion

7 Future Work

Litteratur

[1] Astral Software Inc. uv: An ultra-fast python package installer and resolver. `https://docs.astral.sh/uv/`, 2024. Accessed: 2026-05-30.

[2] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.

[3] Matteo Hessel, David Budden, Fabio Viola, Mihaela Rosca, Eren Sezener, and Tom Hennigan. Optax: composable gradient transformation and optimisation, in jax!, 2020.

[4] Patrick Kidger and Cristian Garcia. Equinox: neural networks in JAX via callable PyTrees and filtered transformations. *Differentiable Programming workshop at Neural Information Processing Systems 2021*, 2021.

[5] Jan Kloppenborg Møller, Marcel Schweiker, Rune Korsholm Andersen, Burak Gunay, Selin Yilmaz, Verena Marie Barthelmes, and Henrik Madsen. *Statistical Modelling of Occupant Behaviour*. A Chapman & Hall Book. Chapman and Hall/CRC, 1st edition, 2024.

[6] Ultralytics LLC. Hidden markov model (hmm) - real-world applications. `https://www.ultralytics.com/glossary/hidden-markov-model-hmm#real-world-applications`, 2024. Accessed: 2026-05-30.

[7] Wikipedia contributors. Akaike information criterion — Wikipedia, the free encyclopedia, 2026. [Online; accessed 31-May-2026].

[8] Wikipedia contributors. Bayesian information criterion — Wikipedia, the free encyclopedia, 2026. [Online; accessed 31-May-2026].

[9] Wikipedia contributors. Likelihood-ratio test — Wikipedia, the free encyclopedia, 2026. [Online; accessed 31-May-2026].

[10] Walter Zucchini, Iain L. MacDonald, and Roland Langrock. *Hidden Markov Models for Time Series: An Introduction Using R*. Number 150 in Monographs on Statistics and Applied Probability. CRC Press, second edition, 2016.

8 Appendix

8.1 Source Code Repository

The repository is hosted on GitHub and the source code can be found there with the data as well: `https://github.com/Maccen26/hmm-project`

8.2 Source code & Envoriment

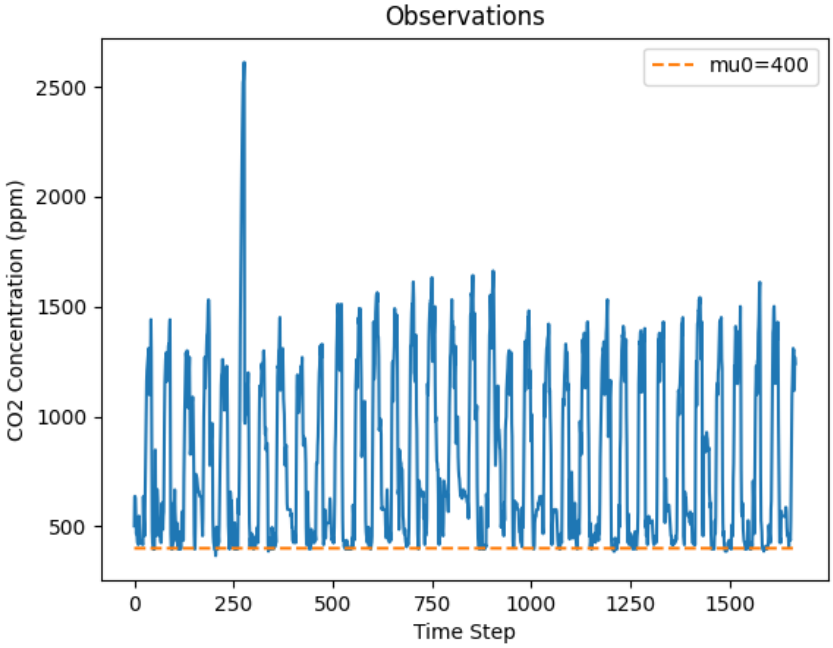
This code was written with Python (version 3.11) and `uv` (version 0.9.7) [1] was used as a package manager. The libraries used are listed in Table 11.

Library	**Version**
`dotenv`	$\geq 0.9.9$
`equinox`	$\geq 0.13.7$
`jax`	$\geq 0.9.0.1$
`matplotlib`	$\geq 3.10.8$
`numpy`	$\geq 1.23.5, < 2.3$
`optax`	$\geq 0.2.8$
`pandas`	$\geq 3.0.0$
`seaborn`	$\geq 0.13.2$
`statsmodels`	$\geq 0.14.6$

Tabel 11: Python libraries used in this project.

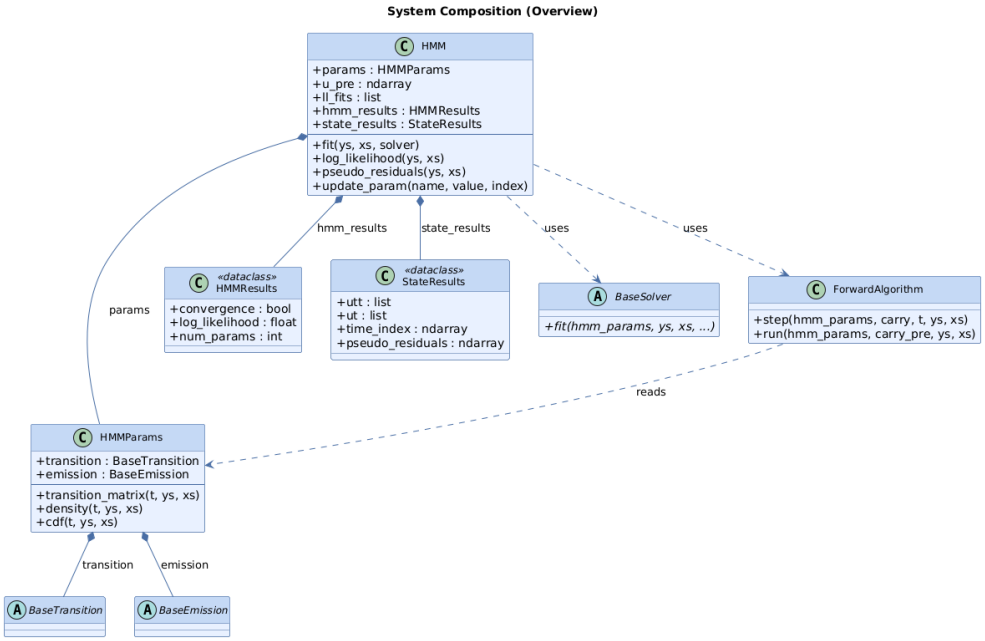
The source code is organized in a way such that the directory `src` contains the source code for the project, the directory `data` contains the data used for training and testing the model, the directory `tests` contains tests for the source code and the directory `drivers` contains scripts for running the model and generating the results used in the report.

8.3 CO_2 Data

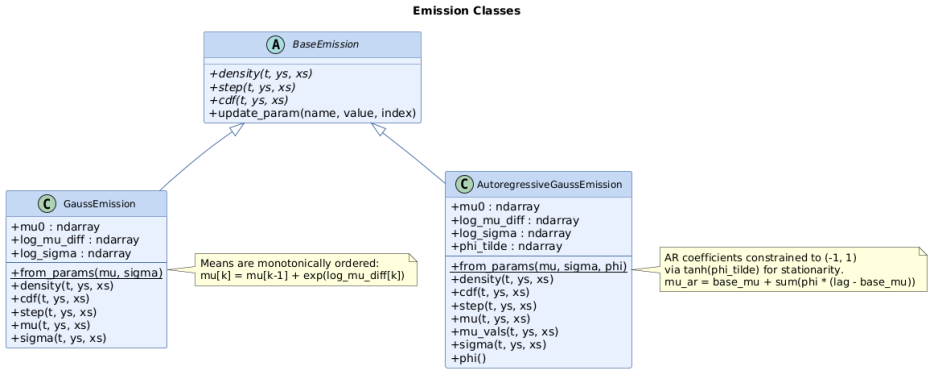


Figur 6: Observed CO_2 concentration (ppm) over time.

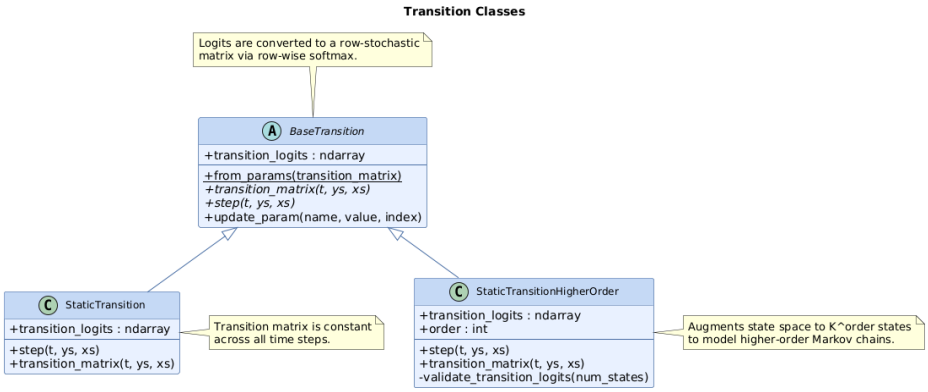
8.4 Class Diagrams



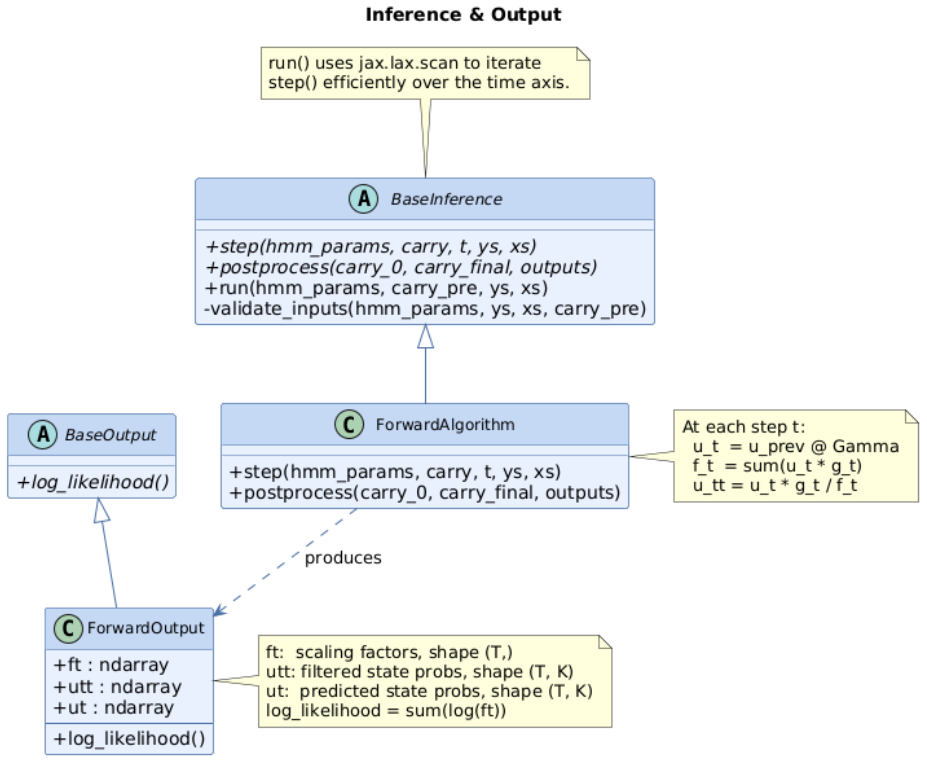
Figur 7: System composition overview. The user-facing `HMM` class owns an `HMMParams` instance composed of a `BaseTransition` and a `BaseEmission`. Fitting is delegated to a `BaseSolver` subclass and state inference is handled by `ForwardAlgorithm`, producing `HMMResults` and `StateResults`.



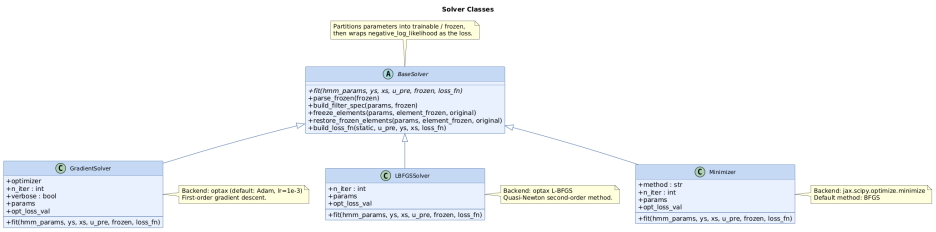
Figur 8: Emission class hierarchy. `GaussEmission` models state-dependent Gaussian distributions with monotonically ordered means enforced via cumulative exponentiation of `log_mu_diff`. `AutoregressiveGaussEmission` extends this with AR(p) dynamics, where coefficients are stored unconstrained (`phi_tilde`) and mapped to $(-1, 1)$ via tanh to guarantee stationarity.



Figur 9: Transition class hierarchy. Both implementations store parameters as logits and convert them to a row-stochastic matrix via row-wise softmax. `StaticTransition` uses a constant $K \times K$ matrix, while `StaticTransitionHigherOrder` augments the state space to K^{order} states to represent a higher-order Markov chain.



Figur 10: Inference and output class hierarchy. `BaseInference.run()` iterates `step()` across the time axis using `jax.lax.scan`. `ForwardAlgorithm` implements the forward filtering pass and produces a `ForwardOutput` storing the scaling factors f_t, filtered state probabilities $u_{t|t}$, and predicted state probabilities u_t. The log-likelihood is computed as $\sum_t \log f_t$.



Figur 11: Solver class hierarchy. `BaseSolver` partitions parameters into trainable and frozen subsets and wraps the negative log-likelihood as the loss function. The three concrete solvers share the same interface but differ in their optimization backend: `GradientSolver` uses first-order gradient descent via `optax`, `LBFGSSolver` uses quasi-Newton L-BFGS via `optax`, and `Minimizer` wraps `jax.scipy.optimize.minimize`.

8.5 Class Design

The `HMM` class (7) is the main entry point of the system. Its responsibility is to act as a wrapper for the other components and to provide a simple interface for training and inference. It is dependent on `Emissions`, `Transitions`, `HMMParams`, and `Solvers`, which are composed together to form a complete HMM.

The `Emissions` (8) and `Transitions` (9) classes are abstract interfaces whose concrete implementations must be provided at runtime. This design choice is motivated by the structure of HMM variants: the differences between models in this project lie in how the density of an observation given a state is computed (emissions) and how the transition probabilities between states are computed (transitions). By separating these concerns into abstract interfaces, different HMM models can be composed without changing the rest of the system.

The `HMMParams` class acts as a container for the model parameters and as a wrapper for the methods implemented in the `Emissions` and `Transitions` classes. This is required by how `JAX` and `Equinox` work: parameters must be stored as pytrees to support automatic differentiation.

The `Inference` class (10) is an abstract interface responsible for implementing inference algorithms such as the forward algorithm, the backward algorithm, and the Viterbi algorithm. In this project, only the forward algorithm is implemented, but the design allows easy extension to other inference algorithms without modifying existing code.

The `Solvers` class (11) implements the optimization algorithms and depends on a loss function that takes the `HMMParams` and `Inference` classes as arguments. This dependency injection of the loss function decouples the optimizer from the objective, making it straightforward to change the loss function without altering the optimization algorithm.